

# Introduction to Python

## Module 3 — Control Flow

Conditionals and loops

## Making Decisions: `if`

Programs need to **react** to different situations. The `if` statement runs code only when a condition is `True`.

```
score = 72

if score >= 60:
    print("Pass")
else:
    print("Fail")
```

**Indentation matters in Python!** The indented lines belong to the `if`. Python uses spacing (not braces) to group code. Be consistent — 4 spaces is standard.

## More Conditions: elif and Comparisons

Chain multiple conditions with `elif` (“else if”). Python checks them in order and runs the *first* match.

```
if score >= 85:  
    grade = "A"  
elif score >= 70:  
    grade = "B"  
else:  
    grade = "C"
```

The **comparison operators**: `==` (equal), `!=` (not equal), `<`, `>`, `<=`, `>=`. Combine conditions with `and`, `or`, `not`. *Careful: `=` assigns a value, `==` tests for equality. Mixing them up is a classic bug.*

## Repeating: for Loops

A for loop runs a block of code **once for each item** in a collection. This is how you process every document, word, or row.

```
words = ["cat", "dog", "bird"]  
  
for word in words:  
    print(word.upper())  
  
# CAT  
# DOG  
# BIRD
```

The variable `word` takes each value in turn. The indented body runs once per item. This is the single most common pattern in data work.

## Looping Tools: range and enumerate

`range(n)` — loop a fixed number of times:

```
for i in range(3):  
    print(i)  
# 0  
# 1  
# 2
```

`enumerate()` — get position *and* item:

```
for i, w in enumerate(words):  
    print(i, w)  
# 0 cat  
# 1 dog  
# 2 bird
```

*There's also the `while` loop, which repeats as long as a condition holds — useful, but `for` covers most needs.*

## A Complete Example: Counting Words

Putting it together — loops, conditions, and a dictionary to count words:

```
text = ["cat", "dog", "cat", "bird", "cat"]
counts = {}

for word in text:
    if word in counts:
        counts[word] += 1
    else:
        counts[word] = 1

# counts is now {"cat": 3, "dog": 1, "bird": 1}
```

*This is essentially what a word-frequency counter does — you built one from scratch!*

## Module 3: Takeaways

- ✓ `if / elif / else` run code based on **conditions**
- ✓ **Indentation** defines what belongs to a block — be consistent
- ✓ Compare with `==` `!=` `<` `>` `<=` `>=`; combine with `and` or `not`
- ✓ `for ... in` loops over every item in a collection
- ✓ `range()` and `enumerate()` are your looping companions

**Next:** Module 4 — **functions**: packaging code so you can reuse it.